

SCIO Clone — Build Plan

Author: Lam Nhat Hao · **Date:** 1 Oct 2026 · **Audience:** the engineering team that will build this, and the leadership who will fund it.

How to read this. Section 1 says what SCIO is in my words. Section 2 is every scoping decision, with the reason. Sections 3–5 are what we build, in what order, and how long. Section 6 is the one thing that surprised me and what it changed. Section 7 is what I would cut first if the number is too big.

1. What SCIO actually is

SCIO is a multi-tenant control plane for screens. A customer pairs a device (a stick, a TV, a laptop) by typing a short pair code into the portal, and from then on the portal decides *what* that screen shows, *when*, in *what layout*, and *how the device itself behaves*. Everything else is in service of that loop.

After using the product and reading the full help center (416 articles, which I scraped for Part 1), the product decomposes into six subsystems. The article counts are a rough proxy for where customer complexity actually lives.

Subsystem	What it covers	Help-center weight
Fleet	Pairing, screens, folders, tags, device settings, remote reboot/screenshot, online status, mass provisioning, operational schedules (power on/off)	~60 articles
Content	Assets (upload, transcode, storage), Websites/URLs, Designer templates, Canva	~30
Composition	Playlists (nested, per-item duration, playback rules), Split-screen zones, Schedules with default content	~20
Apps	100+ integrations: YouTube, Google/Microsoft apps, Social, Weather, News, Stock, Calendar, Dashboards, OptiSync data mapping	~150
Organisation	Users, roles, folder-level permissions, SSO/SAML, billing and plan gating, teams	~30
Players	12+ platforms: Android/Fire TV, Windows, macOS, Linux, Raspberry Pi, ChromeOS, BrightSign, LG webOS, Samsung Tizen, Roku, Apple TV, web browser	~70

Two things that are easy to miss from the outside. First, the player is not a dumb renderer: it caches everything locally, keeps playing when the network drops, buffers proof-of-play events offline and syncs them later. Second, the business model is feature gating by plan (Free → Standard → Pro → Pro Plus → Engage → Enterprise). Almost every "Advanced" article opens with which plan unlocks it. The clone has to model entitlements from day one or retrofitting them touches every endpoint.

2. Decisions the brief left open

I am deciding these, and saying why, so the live discussion can challenge the decision rather than the ambiguity.

Goal. A customer-ready v1 that a 50-location retail chain could run on, not a feature-complete replica. Parity with 100+ apps is a years-long catalogue effort and not where the engineering risk is.

Team: 6 engineers, 9 months. - 1 tech lead (architecture, API, reviews, half-time coding) - 2 backend (API, media pipeline, sync protocol) - 2 frontend (portal, designer, player UI) - 1 player/device engineer (Android shell, caching, device commands) - plus a shared QA/DevOps contractor at 0.5 FTE from month 3
Why 6: the product has four independently deep tracks (API, portal, player, media pipeline). Fewer than 5 means tracks serialise. More than 7 and the tech lead stops coding and the pairing/sync protocol, which is the hardest part, gets designed by committee.

Platforms: web portal, web player, Android player. Android covers Fire TV, Android sticks, tablets, and the OptiStick-class hardware, which is the bulk of real deployments. The web player runs in any browser and gives us a Windows/macOS/Linux player for free via a thin Electron wrapper. Samsung Tizen, LG webOS, BrightSign, Roku, Apple TV are each a separate codebase with vendor certification; they are out of v1.

Stack: TypeScript end-to-end. React + Next.js portal, NestJS API, PostgreSQL with row-level security per tenant, Redis, S3 + CDN for media, MQTT (EMQX) for device push, FFmpeg workers for transcoding. One language means the player, portal, and API share the content schema and the schedule-resolution logic, which is the single most bug-prone piece of logic in a signage product. Choosing Go or Java for the API would be defensible but would fork that logic.

In scope for v1

Area	Included
Fleet	Pair by code, screens, folders, tags, heartbeat/online status, remote reboot, screenshot, device settings, device-level operational schedule (black screen on/off)
Content	Image/video/PDF upload, transcoding, thumbnails, Website/URL asset, a basic Designer (text, image, shape, background)
Composition	Playlists incl. nested and per-item duration, split-screen zones with 4 preset layouts, weekly recurring schedules with date ranges and default content
Apps	An app framework plus 5 reference apps: YouTube, Weather, Google Slides, RSS, Clock
Organisation	Orgs, teams, users, invite, default and custom roles, folder-level permissions, SAML SSO, Stripe billing with plan gating
Players	Web player with offline cache, Android player, Electron desktop wrapper
API	Public REST + GraphQL with API keys, proof-of-play basic report and CSV export

Deliberately out of v1, and why

Cut	Why
The other 95+ apps	Catalogue work, each is 1–3 days once the framework exists; no architectural risk
OptiSync data mapping / repeaters	A product in itself (spreadsheet → template binding engine)
Engage / Kiosk touch, Kiosk Designer Pro	Different interaction model, different plan, different buyer
HDMI-CEC / RS-232 hardware control	Needs owned hardware and per-TV quirks; device-level on/off covers 80 % of the use case

Cut	Why
Tizen, webOS, BrightSign, Roku, Apple TV players	Vendor certification cycles; add one per quarter after launch
AI Camera, Sensors/IoT, OptiSound, Emergency alerts	Niche, each needs partners or licensing
Teams Rooms / Zoom Rooms / Meet hardware discovery	Depends on third-party admin APIs; post-v1
Canva, marketplace, broadcast-to-Teams	Nice-to-have integrations

3. Architecture

3.1 The core loop: resolve, publish, cache, play

The design centre is a **per-screen content manifest**. The server resolves "what should screen X show right now and for the next 24 hours" into a versioned JSON document: schedule → playlist → zones → assets, with every asset as a CDN URL plus hash. The player downloads the manifest and every asset it references into local storage, then plays entirely from cache. A change in the portal bumps the manifest version and pushes a short MQTT message; the player fetches the new manifest and only the assets it does not already have.

Why this shape: - The player never needs the network to keep playing. That is the product's core promise. - The hard logic (schedule resolution, nested playlist flattening, zone assignment) runs once on the server, in one place, in one language, and is unit-testable without a device. - The player is thin and the same code runs on web, Android WebView, and Electron.

3.2 Components

```

Portal (Next.js) —REST/GraphQL→ API (NestJS) —> PostgreSQL (RLS per org)
                                     | | —> Redis (sessions, manifest cache, rate
limits)
                                     | —> S3 + CDN (assets, thumbnails, renditions)
                                     | —> Job queue (BullMQ): transcode, thumbnail,
                                     |      manifest rebuild, proof-of-play rollup,
email
Player (React core) —MQTT (EMQX)←- push: manifest changed, reboot, screenshot, settings
  | Web (browser)
  | Android (Kotlin shell + WebView) —HTTPS→ heartbeat 60 s, proof-of-play batch,
screenshot upload
  | Electron (Win/mac/Linux)

```

3.3 Domain model (the tables that matter)

```

org → team, user, membership(team, role), role(capabilities), folder(kind: screen|asset|playlist,
permission), screen(pair_code, device_info, folder, tags, settings, current_manifest_version),
asset(type, renditions, folder, tags), playlist(items[asset|playlist|app], folder),
layout(zones[]), schedule(rules[], default_content), app_instance(app_type, config),
assignment(screen → asset|playlist|layout|schedule), manifest(screen, version, json),
playback_event(screen, asset, start, end), plan, entitlement.

```

Every row carries `org_id`; PostgreSQL row-level security enforces tenancy below the application layer. This is cheap to add on day one and nearly impossible to add later.

4. Build order and why

Each milestone ends with something a real customer could use. Order is driven by **risk first, then value**: the sync protocol and media pipeline are where unknowns live, so they come before any feature breadth.

#	Milestone	Weeks	Exit criterion	Why here
M0	Foundations	1–3	Monorepo, CI/CD, staging env, auth + orgs + invites, Postgres schema with RLS, S3/CDN, logging and metrics	Everything after this is parallel; nothing can start without tenancy and deploy
M1	A screen shows a picture	3–8	Pair by code, upload image/video, transcoding, assign to screen, web player with cache, heartbeat and online/offline	Proves the manifest/cache/push loop end to end. If this is wrong, nothing else matters
M2	Playlists and schedules	8–14	Playlists (nested, durations), weekly schedules with default content, folders, tags, Android player	The first thing a customer actually buys. Android here because it is the real-world device and exposes caching and autostart issues early
M3	Layouts, designer, apps	14–20	Split-screen with 4 layouts, basic designer, Website asset, app framework + 5 apps, Electron wrapper	Composition breadth. App framework is a sandboxed iframe with a config schema, so each later app is a day of work
M4	Fleet and organisation	20–28	Roles + folder permissions, remote reboot/screenshot/settings, operational schedule, proof-of-play basic, public API + keys, mass provisioning	Needed for multi-location buyers. Deliberately after M3 so the permission model can be tested against real resource types
M5	Harden and launch	28–36	SAML SSO, Stripe billing + plan gating, load test to 10 000 screens, pen test, docs, support runbooks	Gating is enforced at the API layer on entitlements that already exist in the schema; this phase wires UI and billing to them

Why not apps first? They are the most visible feature and the biggest share of the help center, but they sit on top of the asset model and the player. Building them earlier means rebuilding them when the player changes.

Why permissions in M4 and not M0? Roles and the folder permission model need real folders of real resource types to test against. In M0 we lay the schema (`folder.permission`, `membership.role`) and enforce admin-only; the fine-grained model lands when there is something to be fine-grained about.

5. Estimate

Effort in engineer-weeks (ew), then checked against team capacity. Numbers are from the milestone breakdowns below; each line is something I can defend individually.

Milestone	Backend	Frontend	Player	Lead	Total ew
M0 Foundations	5	3	1	3	12
M1 Screen shows a picture	10	7	8	3	28
M2 Playlists, schedules, Android	10	10	10	3	33
M3 Layouts, designer, apps	8	16	6	3	33
M4 Fleet and org	16	12	8	4	40
M5 Harden and launch	10	6	4	4	24
Total	59	54	37	20	170 ew

Capacity: 5.5 engineers × 36 weeks = **198 ew**, plus 0.5 QA/DevOps from month 3. That is a **16 % buffer**, which is thin for a greenfield product but honest. Most of the 28 ew of slack sits deliberately in M5: hardening is where slips from M1 to M4 land, and the launch date is what leadership will hold us to. The biggest single risks to the number, in order:

1. **Android device variance** (M2). Autostart, WebView versions, storage quotas and scaling bugs differ by vendor. Help-center evidence: three separate articles on autostart failures across Fire TV and Android TV, and two on Android display scaling alone. Mitigation: buy 6 reference devices in week 1 and run the player on all of them from M1.
2. **Media pipeline** (M1). Transcoding, resolution ladders, and "why does my video stutter" are a permanent support load. Mitigation: normalise everything to H.264 1080p30 plus a 720p rendition; refuse exotic codecs with a clear error rather than trying to play them.
3. **Designer scope creep** (M3). A canvas editor can absorb unlimited time. Mitigation: four element types, no animation, no layers panel in v1.
4. **Schedule resolution edge cases** (M2). Timezones per screen, overlapping rules, default content, DST. Mitigation: pure function, property-based tests, resolved server-side only.

If leadership needs a smaller number, Section 7 lists what to remove and what each removal saves.

6. The thing I did not expect, and what it changed

Content access is decided by folders, not by per-item permissions. I assumed a signage portal would let you share an individual screen or playlist with a named user, the way a file is shared in Google Drive. I checked this in the app: the Change Permissions dialog exists on folders in Screens, Files/Assets and Playlists, and a single asset has no such option. Instead, SCIO layers three things: Teams (a workspace whose screens, assets and users are invisible to other teams), Roles (seven defaults plus custom roles that say what a user may do), and then puts the real access control over content on *folders*: a Screens folder set to Admin Only, an Assets folder restricted to the Department A users, a Playlists folder restricted to Department B. The help center's own walkthrough for "let two departments each edit only their zone of a shared screen" is a sequence of folder-permission settings plus a split-screen layout, nothing more. There is no per-asset permission at all; the help center says so outright, and the whole stack is gated to the Pro Plus plan.

Once I saw it, it made sense: the customers who need permissions are multi-location operators, and they think in locations and departments, which map naturally onto folders. A resource-level ACL would be more powerful and much harder to explain to a store manager.

What it changed in the plan

1. **Folders became a first-class entity in the M0 schema**, with `kind` and `permission`, rather than a UI-only grouping added in M4. Every screen, asset, and playlist belongs to exactly one folder from the first migration.
2. **The permission model shrank**. I had budgeted ~6 ew in M4 for a resource-level ACL with policy evaluation. Folder-level inheritance plus a capability set per custom role is ~3 ew. The saved time moved to the Android player, where the help center shows the real pain is.
3. **The public API has to expose folders**, because any integration that creates screens or uploads assets has to say where they go. That was not in my first API sketch.

A second surprise, smaller but relevant: the help center documents **two portal UIs side by side** ("1.0" and "Updated"), with every step shown twice. The company is mid-migration of its own frontend. For the clone, that is a strong argument for keeping all logic in the API and the player, and treating the portal as a thin, replaceable client. It confirmed the TypeScript-everywhere, logic-in-the-API decision in Section 2.

7. If the number has to be smaller

In the order I would cut, with what each cut saves and what it costs the customer:

Cut	Saves	Customer impact
Electron desktop wrapper	3 ew	Windows/mac users use the browser player
Designer (keep Website/URL + Canva embed as the "design" path)	10 ew	Customers design in Canva or Slides, which many already do
Proof-of-play	6 ew	Pro Plus feature; no v1 analytics
Public API (keep internal only)	6 ew	Blocks enterprise integrations, fine for SMB launch
Reduce apps from 5 to 2 (YouTube, Website)	4 ew	Framework stays, catalogue grows later
SAML SSO	4 ew	Enterprise-only

Cutting all six takes the plan to **~137 ew, about 7 months with the same team**. I would not cut below that: M0–M2 plus split-screen plus folder permissions is the minimum a paying multi-location customer needs.

8. First two weeks, concretely

1. Buy reference devices: Fire TV Stick 4K, two Android sticks of different vendors, an Android tablet, a Chromebox, a Raspberry Pi 4.
2. Monorepo (`apps/portal`, `apps/api`, `apps/player`, `packages/schema`, `packages/schedule-engine`), CI, staging on one cloud account.
3. Schema migration 001: `org`, `team`, `user`, `membership`, `role`, `folder`, `screen`, `asset`, `playlist`, `schedule`, `manifest`, `plan`, `entitlement`. RLS policies on all of them.
4. Spike: pair code flow end to end with a hard-coded image, MQTT push, web player cache. Two engineers, one week. Its result decides whether the manifest design in 3.1 survives.

5. Write the schedule-resolution function as a pure library with tests before any UI touches it.